

Foundations of hardware verification

Slide 1 · 0–2 min

Foundations of hardware verification

A passing test is evidence with a defined scope.

Requirement → Observation → Claim

Teaching script

Introduce the course through the question: what does a passing test entitle us to claim? Ask learners to keep one recent verification decision in mind. The lesson ends by revisiting that decision with an explicit checker and closure criterion.

Distinguish functional verification before fabrication from validation on real hardware and manufacturing test of physical defects. This is the book's terminology; other literature uses validation more broadly.

Source: ch01:11–19, ch03:97–146

Learning outcomes

Slide 2 · 2–5 min

Learning outcomes

Make a verification claim that another engineer can audit.

- Distinguish verification, validation and manufacturing test.
- Identify shared assumptions between design and checker.
- Select an oracle and state its limits.
- Demand evidence for a stage exit.

Teaching script

Ask each learner to select the outcome that is least explicit in their current work. These are observable outcomes: a definition, a dependency analysis, a checker choice and an evidence request.

Revisit the list at the end rather than asking whether the audience understood. A learner should be able to demonstrate each outcome on the reference DMA.

Source: ch01:11–19, ch02:11–57, ch03:97–146, ch04:168–199

Route through the lesson

Slide 3 · 5–7 min

Route through the lesson

Four questions connect the foundations.

- Why do escapes survive?
- Where can independent checking fail?
- What can a test actually establish?
- When is a block ready for its next use?

Teaching script

The course allocation is ninety minutes: introduction, four discussion blocks, an exercise and closure. These timings are teaching allocations, not measured engineering durations.

Explain that the detailed handout retains the wider examples. The projection intentionally selects a short argument, so an omitted slide is not an omitted study topic.

Source: ch01:11–19, ch02:11–57, ch03:9–67, ch04:11–48

Scope before method

Slide 4 · 7–11 min

Scope before method

Different checks answer different questions.

Model versus specification → Real hardware behavior → Manufactured die defects

Teaching script

Ask for a concrete example of each task. Checking an RTL model against the specification is functional verification. Checking the implementation on real hardware is validation in this book. Identifying a physical defect in a manufactured die is manufacturing test.

Avoid presenting these as isolated organizations. Their evidence feeds later decisions, but their reference objects and failure mechanisms differ.

Source: ch01:11–19

A bug escape crosses a boundary

Slide 5 · 11–16 min

A bug escape crosses a boundary

Block correctness does not automatically compose.

Standalone blocks → Shared resources → Integration checks

- Correct blocks can interact incorrectly.
- Arbitration, ordering and back-pressure emerge during integration.

Teaching script

Use the reference SoC: a neural accelerator streams weights while a DMA fills the next buffer. Each block can pass alone while the shared interconnect introduces ordering and arbitration behaviors absent from either standalone environment.

Ask learners which observable event would reveal an integration failure. Require an observation and a checker, not simply more traffic. Preserve the distinction between an escape to integration and an escape to silicon.

Source: ch01:11–19, ch01:60–122

Read the denominator

Slide 6 · 16-21 min

Read the denominator

Survey headlines can measure different outcomes.

- First-silicon success concerns IC/ASIC fabrication.
- FPGA field escapes concern shipped systems.
- Scope and uncertainty belong beside the percentage.

Teaching script

The book explicitly warns against directly comparing first-silicon success with non-trivial FPGA bugs in production. They are different outcome measures, and the gap between their headline percentages is smaller than the stated survey margin.

Explain that effort figures also depend on scope: pre-silicon verification, a broader validation definition, time and headcount are different denominators. Ask which scope a learner would request before reusing an effort estimate.

Source: ch01:20-43, ch01:44-59

Discussion: move verification to the lab?

Slide 7 · 21–25 min

Discussion: move verification to the lab?

A cheap reprogramming step does not make an escape cheap.

- An FPGA schedule cuts pre-lab verification.
- Which new behavior still needs model-level checking?
- What observability would the team lose?

Teaching script

Give pairs one minute to propose a targeted compromise. The expected answer preserves checks for newly modified behavior while recognizing credible evidence for unmodified verified IP.

Debrief the difference between reprogramming cost and debugging cost. The chapter's scenario retains verification of a new camera datapath while reusing established UART and SPI evidence. Do not infer that all reused IP can skip integration checking.

Source: ch01:20–43, ch01:60–122

Independence starts at interpretation

Slide 8 · 25–29 min

Independence starts at interpretation

Separate implementations can share the same wrong assumption.

Specification → Independent interpretations → Comparison

- A checker copied from the RTL can reproduce its mistake.
- Independent interpretation must reach the comparison.

Teaching script

Have learners draw the design path and checking path from the specification. Different people or files do not establish independence if both paths inherit the same algorithm or unresolved interpretation.

Use the DMA stride ambiguity from the chapter: a designer and scoreboard author can agree on when the stride applies and still both be wrong. A passing regression then establishes agreement between those interpretations.

Source: ch02:11–57, ch03:97–146

Find the reconvergence point

Slide 9 · 29–34 min

Find the reconvergence point

Shared machinery can remove the independence a test needs.

- Trace what the design and checker reuse.
- Ask which defect that reuse would hide.

Teaching script

Draw a fork and a join. The important question is where the checking path reconnects with a design-derived implementation. If the same incorrect transformation feeds both expected and actual values, a clean comparison is predictable.

Ask for a bounded remedy: review or independently implement the relevant transformation, or use an orthogonal property that does not inherit it. Do not demand total independence of every tool in the organization.

Source: ch02:11–57

Stress the contract

Slide 10 · 34–39 min

Stress the contract

A verifier should expose violations instead of repairing them.

Stimulus → Unmodified observation → Contract check

- Exercise legal corner conditions.
- Keep the observer strict.
- Do not normalize a faulty response into an acceptable one.

Teaching script

Contrast a cooperative client with a verification environment. A real client may retry or compensate; a checker must retain evidence of the initial contract violation.

Discuss invalid stimulus carefully. The chapter does not permit filtering every invalid case away: some invalid inputs have specified handling that must be checked. Conversely, unconstrained nonsense outside the contract needs a clearly stated purpose.

Source: ch02:58–134

Discussion: whose expected value?

Slide 11 · 39–43 min

Discussion: whose expected value?

An agreement can be a common-mode failure.

- The scoreboard uses the designer's address helper.
- All comparisons pass.
- Which failure could remain invisible?

Teaching script

Ask learners to identify the shared dependency before proposing a fix. Expected answer: an incorrect address transformation inside the common helper can appear on both sides of the comparison.

Accept independent specification interpretation or a relevant orthogonal invariant as remedies, provided learners identify what each would detect. Reject “add more seeds” as a sufficient answer to a shared deterministic error.

Source: ch02:11–57, ch03:97–146

The space exceeds enumeration

Slide 12 · 43–47 min

The space exceeds enumeration

Configuration, internal state and history all matter.

Configuration → Internal state → Input history

- An abstract model removes detail; it does not prove omitted behavior irrelevant.

Teaching script

Walk through the chapter's state-space argument qualitatively. Register settings form only the shallow part: FIFO contents, transaction state, multiple active channels and input histories enlarge the problem.

Point to the detailed tables in the handout for arithmetic. Do not interpret the count as a count of reachable or equally likely bugs; the chapter separates potential configurations from reachable histories.

Source: ch03:9–67

Place checks where failures are observable

Slide 13 · 47-52 min

Place checks where failures are observable

Controllability and observability determine useful verification levels.

- A feature must be provoked and its failure seen.
- Moving a check upward can destroy either capability.

Teaching script

Use the SIMD saturation-flag example. Ask what changes when a direct core-level observation is replaced by a firmware path through a whole SoC. The question is the concrete mechanism that makes the defect hard to trigger or to observe.

Connect the answer to allocation: the best level is not automatically the highest or lowest level. Select the level at which both the required stimulus and an effective observation exist.

Source: ch03:68-96

Use complementary oracles

Slide 14 · 52–57 min

Use complementary oracles

Protocol legality does not establish payload correctness.

Protocol property → Functional comparison → End-of-test accounting

- Assertions check bounded statements.
- A scoreboard checks predicted outcomes.
- Completeness checks catch missing transactions.

Teaching script

The DMA property about burst boundaries can establish a protocol condition while legal bursts still carry wrong data. The functional scoreboard compares effects against an independent prediction.

Ask what an end-of-test leftover means: expected work can remain unmatched even if every received response looked legal. Explain that lack of an assertion failure may also reflect a condition that was never activated.

Source: ch03:97–146

Discussion: a clean run with no comparisons

Slide 15 · 57-61 min

Discussion: a clean run with no comparisons

Activity and correctness require different evidence.

- The regression reports no failure.
- No output transaction was compared.
- What claim can the report support?

Teaching script

Let the audience articulate the narrowest defensible claim. A lack of reported failure says little about functional correctness if the path that would detect incorrect output did not operate.

Ask for one additional measurement and one checking mechanism. A comparison count can expose inactivity, but a count alone does not prove the comparison semantics are right. The mechanism must be tied to the intended behavior.

Source: ch03:97-146, ch03:147-213

Plan the evidence before implementation

Slide 16 · 61–65 min

Plan the evidence before implementation

A test list is not a closure argument.

- Name the feature and its checker.
- Define closure evidence and ownership.
- Review assumptions before building the environment.

Teaching script

Use a plan row rather than a schedule bar. Ask what evidence would distinguish “test implemented” from “behavior verified”. The row needs a requirement or feature, a checking mechanism and a criterion that can be assessed.

Planning also interrogates an incomplete specification. Record ambiguity as work requiring resolution rather than burying a chosen interpretation inside a sequence.

Source: ch04:49–88

Bring-up is a checked transaction

Slide 17 · 65–70 min

Bring-up is a checked transaction

Traffic without an active oracle is not verification bring-up.

Drive → Observe → Check

- Demonstrate stimulus, observation and checking together.
- Then grow regression and coverage maturity.

Teaching script

Explain the bring-up exit using one complete path through the environment. A waveform showing activity is insufficient if the predictor, observer or comparison path is disconnected.

As the environment grows, repeat the argument for new features. The lifecycle contains feedback: coverage and bugs change the plan and stimulus rather than simply advancing a fixed sequence of completed tasks.

Source: ch04:89–132

Maturity is demonstrated

Slide 18 · 70–75 min

Maturity is demonstrated

Stage labels summarize checkable conditions.

- Implemented stimulus and checkers.
- Coverage and sustained regression evidence.
- Reviewed exclusions and open-bug dispositions.

Teaching script

Distinguish a block ready for other teams to build on from a block ready for sign-off. The chapter's maturity criteria deliberately use demonstrations instead of subjective completion percentages.

For closure, ask who reviewed exclusions and who accepted the disposition of every open bug. A clean recent run does not replace a sustained regression window or an explicit risk acceptance.

Source: ch04:133–167, ch04:168–199

Discussion: ready for integration?

Slide 19 · 75–79 min

Discussion: ready for integration?

The consumer needs a defensible entry criterion.

- The block generates traffic but its checker is unfinished.
- Another team wants to build on it.
- Which maturity evidence is missing?

Teaching script

Ask learners to answer using a missing demonstration, not a mood or a completion percentage. The block has not established the implemented checking and regression evidence expected of verification maturity.

Discuss a conditional integration decision as an explicit risk decision with owners and limits. It must not silently redefine the maturity label for one block.

Source: ch04:133–167, ch04:168–199

Exercise: interpret a DMA pass

Slide 20 · 79–85 min

Exercise: interpret a DMA pass

State exactly what the oracle established.

- No burst-boundary assertion failed. No other failure was reported.
- Give the two readings of this result.
- Name the additional oracle and its end-of-test check.

Teaching script

Assign book Exercise 3.4. Allow two minutes individually, two minutes in pairs and two minutes for debrief. The prompt comes directly from the chapter's partial-oracle example.

Expected answer: the boundary property may have been satisfied on exercised behavior or may not have been activated. Neither reading establishes data correctness. An independent functional scoreboard checks the predicted effects; its end-of-test accounting detects expected or unmatched transactions left over.

Assessment: a complete answer separates activation from satisfaction, names the functional comparison and explains the leftover check. More simulation time without an active checking path is not an adequate answer.

Source: ch03:97–146

What to change on Monday

Slide 21 · 85–87 min

What to change on Monday

Make one existing claim auditable.

- Trace one expected-value dependency.
- Check whether one claimed behavior reaches an active oracle.
- Replace one vague milestone with observable evidence.

Teaching script

Invite each learner to choose one action that can be demonstrated in their own environment. Keep the scope to an existing claim or plan row; this is not a mandate to rebuild the testbench.

Ask what artifact they would bring to a review: a dependency diagram, an observed comparison with an injected failure, or a plan row with closure evidence.

Source: ch02:11–57, ch03:97–146, ch04:49–88

Scope and next reading

Slide 22 · 87–90 min

Scope and next reading

The foundations guide later technique choices.

- Read the Foundations handout and book Chapters 1–4.
- Next: verification planning, coverage and closure.
- This lesson omits tool setup, UVM implementation and formal algorithms.

Teaching script

Revisit the four learning outcomes using short responses from the room. Ask learners to state one limitation of the oracle they currently rely on most.

Direct learners to Chapters 1–4 and the retained Foundations handout for detailed examples and citations. Later lessons develop methods; this session established the questions those methods must answer.

Source: ch03:147–213, ch04:49–88